solar-sync-broker — arhitectura țintă (ce trebuie să fie la final)

Doar Docker: docker compose pe EC2, noțiunile din car-platform traduse fără Kubernetes · F = feature-ul din PLAN.pdf · eticheta = profilul compose · linie punctată = partea III–IV (F11–F15)

Browser (Internet)

Cloudflare DNS
*.solarsync.<domeniu>
A record → EC2 EIP

EC2 Elastic IP
SG: 80, 443, 22 (doar IP-ul tău)

Clienți MCP (F7)
Claude Desktop · MCP Inspector

Model AI (F10) · extern
apelat de LlmPlanner din agent · cheia în .env

GitHub · monorepo-ul solar-sync-broker
Actions: test → build-push → update-compose → update-platform → deploy (ssh)

Docker Hub · imagini sha-… (multi-arch)

EC2 · Docker Engine · docker compose (solar-sync-broker-ops/docker-compose-platform.yaml)

edge · rețeaua Docker *edge* · singurele porturi publicate: 80 / 443

nginx [edge]
TLS Let's Encrypt (certbot)
app · api · mcp · auth
grafana · kibana · kafka-ui

app

ui · Angular (U1–U4) [core]
fișiere statice pe nginx
login: F1 → Keycloak PKCE (U4)

/api

Kong — API gateway (F15) [edge]
DB-less: kong.yml declarativ în Git
/api/v1 · /mcp → api · singura intrare spre aplicații
jwt (Keycloak) · rate-limiting · correlation-id · prometheus

oauth2-proxy (F14) [edge]
login Keycloak pentru UI-uri
grafana · kibana · kafka-ui
doar grupul /admins

OIDC

grafana · kibana · kafka-ui

mcp.* + token

MCP prin Kong: citire + proposePlan

auth (F14)

Keycloak [auth]
start --import-realm

Postgres keycloak-db [auth]
volum + healthcheck

realm-solar-sync.json în Git · roluri USER / OPERATOR / ADMIN / AGENT · clienți: api (resource server) · agent (client credentials) · oauth2-proxy

aplicațiile tale · rețeaua Docker *backend* · fără porturi publicate: intră doar trafic de la Kong (F15)

solar-sync-broker-api · monolitul (F1–F11) [core]

REST /api/v1 (F1–F6, F8)

Server MCP · Spring AI (F7–F8)

Spring Security · JWT (F1) → **resource server pe Keycloak** (F14) · hasAuthority = matricea

auth·users (F1) | house (F2) | **battery** poarta (F3) | simulare tick (F4) | financial (F5) | grid (F6) | agent runs (F8)

Producer **telemetry.recorded · grid.state** (F11)

Consumer **measurements** (F12) · @KafkaListener financial, grid (F11)

Logback JSON pe stdout + MDC tickId/houseId (F4) · Actuator + Micrometer → /actuator/prometheus (F6) · OpenTelemetry OTLP → Tempo (F13)

aprobarea unui plan cheamă executeCommand din battery: aceeași cale și aceeași poartă pentru comanda manuală și pentru cea a agentului

solar-sync-broker-agent · primul microserviciu (F9–F10) [core]

Client MCP · cont AGENT (F9) → client credentials (F14) · prin Kong (F15)

RuleBasedPlanner (F9)
prag 24 h · 0,75 / 1,4

LlmPlanner (F10)
Spring AI ChatClient → Model AI

fără bază de date · fallback pe reguli · ascultă grid.state · runId în MDC

consumă

solar-sync-broker-simulator (F12) [messaging]

PvGenerator · LoadGenerator · PriceGenerator · SimulationClock
sămânța din .env · fără bază de date

publică

grid.state

messaging (F11)

Kafka apache/kafka · KRaft, un nod · volum [messaging]

topic **measurements** · simulator → api

topic **telemetry.recorded** · api → financial, grid

topic **grid.state** · api → agent

kafka-init (creează topicurile) · **Kafka UI** (kafbat, prin oauth2-proxy) [messaging]

topicurile se creează dintr-un container de init, nu din cod · consumatori idempotenți: UQ (house_id, recorded_at) în bază

data · volume Docker numite

MySQL 8.4 · solar_db [core] · JPA din api
Flyway V1–V8 · ddl-auto: validate · healthcheck → depends_on: service_healthy

Postgres-ul Keycloak e în profilul auth · parolele doar în .env pe EC2, în Git doar .env.example

monitoring (F6 · F13)

Prometheus [obs] | Grafana [obs]

Alertmanager [obs] | Tempo (F13) [obs]

prometheus.yml scrape: api, agent, simulator, Kong · Grafana provisioning din Git · dashboard „SCADA" · alertă: rețea CRITICAL

logging (ELK · F13)

Elasticsearch [logs] | Kibana [logs]

Filebeat · citește /var/lib/docker/containers [logs]

logurile JSON din toate containerele → Elasticsearch → Kibana · căutare după tickId / traceId

Ce ține locul Kubernetes (ca la cars, fără cluster)

rețele Docker edge · backend · obs
Kong și oauth2-proxy sunt în ambele = „NetworkPolicies"

profiluri compose
core · messaging · auth · edge · obs · logs

healthcheck + depends_on service_healthy
restart: unless-stopped · limite de memorie

config în Git: kong.yml · realm.json · prometheus.yml
grafana/ · filebeat.yml · nginx.conf · .env.example

Memoria EC2: profilurile core + edge + auth + obs pe o instanță medie; messaging și logs (Elasticsearch ~1–2 GB) pe una mai mare sau doar local · Local: același compose, cu profilurile pe care le pornești

Fluxul de livrare (ce ai deja în CI, dus până la capăt): 1. git push → 2. test (mvn verify) → 3. build-push imagine sha-… în Docker Hub → 4. update-platform scrie tag-ul în docker-compose-platform.yaml → 5. deploy: ssh pe EC2 → docker compose pull && up -d

Legendă
portocaliu = edge / gateway (Cloudflare, EC2, nginx, Kong, oauth2-proxy) · roșu = Keycloak și logurile · mov = metrici și tracing · verde = Kafka · galben = date · albastru = Docker în locul Kubernetes · violet MCS = aplicațiile tale
Partea V: U1–U4 interfața Angular (după F3, F6, F8; U4 între F14 și F15) · **Partea IV din PLAN.pdf**: **F14** Keycloak (realm în Git, API = resource server, agentul pe client credentials, oauth2-proxy pe UI-uri) · **F15** Kong = API gateway (jwt pe Keycloak, rate-limiting, correlation-id, metrici; agentul trece și el prin Kong) + rețelele Docker: aplicațiile fără porturi publicate